What Do Neural Predictors Add to Budget Pacing?

Daily Totals, Hourly Shape and Hedging in Throttling-Based Pacing on Criteo Campaign Logs

Mai Minh Duc
FPT University
Ho Chi Minh City, Vietnam
maikk19funi@gmail.com

Abstract

Demand-side platforms pace daily campaign budgets by throttling participation in auctions. A natural way to pace is to forecast the demand of the remaining day and throttle so that the forecast fits the remaining budget. We ask what neural predictors add to it. On 16,468,027 real impressions from 414 campaigns in the Criteo Attribution dataset, we couple rolling-origin probabilistic forecasts from two neural predictors (NHITS and the Temporal Fusion Transformer, TFT) and two non-neural references (seasonal naive and LightGBM) with an impression-replay pacing environment and compare them with no pacing, even pacing and a proportional-integral-derivative (PID) controller tuned on validation days, at three budget levels. Throttling on the median neural forecast captures 3.4–5.4 points more clicks than even pacing and 0.7–2.0 points more than tuned PID. Over a seasonal-naive profile the neural models add 1.2–2.1 points, with 1.4–1.9% more spend and no loss of clicks per unit of spend. One scale on the naive forecast total, chosen on validation days, recovers 36% to 94% of the TFT gain depending on the budget. In a swap of daily total and hourly shape, the forecast total supplies 68–82% of the gain, and an exploratory matched-spend comparison leaves TFT and NHITS 0.47–1.20 and 0.44–1.30 points above what scaling alone reaches. An oracle forecast shows 1.03–1.54 points of headroom over TFT. Planning against the 90th percentile costs 7.7–10.6 points. Summing hourly quantiles overcovers the day (94.0–97.4% against a nominal 90%), but a calibrated daily-total hedge recovers only 11–20% of that loss for TFT, and PID feedback on top of the forecast helps at the tight budget only. This simulator barely penalizes early depletion, so part of the hedging cost is a property of the environment. Forecast throttling stays 3.9–6.7 points ahead of a tuned dual-gradient pacer. A synthetic world of our own shows the same ordering of the pieces, but a small replication on a second real dataset (iPinYou, 5 advertisers, 10 test episodes) is descriptive and does not reproduce it: a 3-day replay forecast is no better than even pacing, while an oracle is 4.9–6.4 points ahead. The main results come from 5 test days of one dataset and a throttling-only simulator. We will release the code, protocol and results files.

Keywords

budget pacing, throttling, real-time bidding, probabilistic forecasting, neural predictors, empirical study

1 Introduction

Advertisers buy display impressions with daily budgets, and the platform must spend each budget across the day rather than exhaust it in the first busy hours [1, 11]. A mechanism used in production systems is *probabilistic throttling*: the platform takes part in each eligible auction with a probability α_t that it adjusts over the day [1, 2, 16]. How to set α_t is the pacing problem.

Forecast-free controllers, such as even pacing or a proportional-integral-derivative (PID) loop on the spend gap, react to past spend. A forecast-driven policy instead predicts the remaining day's eligible spend and sets α_t so that the predicted spend fits the remaining budget. Modern multi-horizon forecasters, including the Temporal Fusion Transformer (TFT) [12] and NHITS [3], produce multi-horizon quantile forecasts that can feed such a policy. A practitioner therefore faces three questions. Does forecasting help over a well-tuned controller? If it does, what exactly does the forecast add: a better daily total, a better hourly shape, or something else? And should the policy hedge by planning against an upper quantile of demand?

We answer these questions empirically on real campaign logs, with neural predictors as the object of study: NHITS and TFT are two instances of the class, and TFT is the reference model in the analyses that need one (the analyses that were also run for NHITS give the same picture). This is an applied study: the forecasters and the throttling model are established methods, used as tools. Our contributions are:

(1) A reproducible pipeline that couples hourly rolling-origin probabilistic spend forecasts with an impression-replay throttling environment built from the public Criteo Attribution dataset [6], and a controlled comparison of forecast-free and forecast-driven policies at three budget-tightness levels.
(2) A decomposition of the gain of a learned forecaster. Over a seasonal profile it is extra budget delivered, not cheaper clicks; a single scale on the naive forecast total recovers a large part of it; a swap of total and shape and an oracle forecast show where the rest lies; and an exploratory comparison at matched spend leaves a smaller edge.
(3) An analysis of hedging: the summed hourly quantiles overcover the remaining day, a calibrated daily-total hedge recovers only part of the loss, and PID feedback on top of the forecast helps only at the tight budget.
(4) Two further environments: a synthetic world from a world builder we developed for this study, in which the same ordering of the pieces appears (Section 6.6), and a small replication on iPinYou that does not reproduce it (Section 6.7).

The design and the main metrics were fixed in a written protocol before the test split was evaluated. The decomposition, hedging and feedback analyses were added after the main results were seen; each new simulation was entered in the protocol log before it ran (Section 5). Every number reported here is generated from results files written by the accompanying code, which we will release.

2 Related Work

Budget pacing. Lee et al. [11] study bid optimization with smooth budget delivery. Agarwal et al. [1] describe pacing at LinkedIn that uses global supply patterns to spread spend. Smart pacing [16] learns per-segment pacing rates to also improve performance. Dai et al. [5] adjust probabilistic throttling through dual multipliers over a percentile space and report an online test on the Taobao brand advertising platform. Zhang et al. [17] use feedback control to adjust bids toward KPI targets such as the winning ratio and the effective cost per click; PID controllers of this kind are a natural forecast-free reference, and ours adjusts the participation rate instead of the bid. Balseiro et al. [2] compare throttling with other budget-management strategies in equilibrium. Optimal bidding under budgets in real-time bidding is studied by Zhang et al. [18]. A recent preprint [14] calibrates forecast uncertainty by its effect on pacing decisions with split conformal prediction, over a catalog of 12 policies and replays calibrated to Criteo Uplift and KuaiRand; among its findings, the forecaster with the lowest error is not the one with the smallest decision radius. A practical guide [4] surveys throttling, PID, model-predictive and dual online-gradient pacing; it notes that a PID controller tracking a target built from a supply forecast cannot by itself correct a misspecified target, and suggests re-forecasting the remaining supply, which is what the forecast throttle here does. It reports no empirical comparison. These works design pacing mechanisms or uncertainty sets. We instead hold the mechanism fixed and ask what the forecast inside it contributes, and where that contribution comes from.

Forecasting. TFT [12] combines recurrent encoding, attention and variable selection for multi-horizon quantile forecasts. NHITS [3] uses hierarchical interpolation with multi-rate sampling. Gradient-boosted trees such as LightGBM [9] are strong tabular baselines. We use the `neuralforecast` implementations [13] rather than custom code, and we evaluate accuracy with MASE [8] and the pinball loss [10].

3 Pacing Setup

Episodes and budgets. An episode is one campaign-day of 24 hours. Campaign c receives the daily budget $B = \rho\,\bar{D}_c$, where $\bar{D}_c$ is its mean daily spend over the training days and $\rho \in \{0.3, 0.5, 0.7\}$ sets the budget tightness. In 94.3%, 86.5% and 70.8% of test episodes, respectively, the day's logged demand exceeds B, so the budget binds.

Throttling environment. At the start of hour h the policy sets $\alpha_h \in [0, 1]$. Each logged impression of that hour is kept independently with probability α_h. Kept impressions are served in log order, with their logged cost, click and conversion, until the next impression would exceed the remaining budget. The uniform draws are fixed per replay seed, campaign and hour, so all policies face identical randomness. We assume that throttling does not change the price or value of the kept impressions. This is a common simplification in throttling-based pacing, and we make no claim about bid-shading interactions. The log contains only impressions that were displayed, that is, impressions won by the logging bidder under its own bidding and pacing. We treat this stream as the eligible demand. It is therefore shaped by the logging policy, and a replay that throttles it further is a counterfactual that the log cannot verify.

Policies. Let S_h be the spend before hour h and $R_h = B - S_h$.

- *No pacing*: $\alpha_h = 1$ until the budget is exhausted.
- *Even pacing*: starts at $\alpha_0 = \min(1, B/\bar{D}_c)$. It then corrects multiplicatively toward an equal spend of $R_h/(24-h)$ per remaining hour.
- *PID*: $\alpha_h = \alpha_0 + K_p e_h + K_i \sum_{j \le h} e_j + K_d(e_h - e_{h-1})$ on the normalized gap $e_h = (Bh/24 - S_h)/B$. The gains are grid-searched on validation days separately for each ρ.
- *Forecast throttle*: at every hour, a fresh forecast $\hat{y}_h^{(q)}, \ldots, \hat{y}_{23}^{(q)}$ of the q-quantile hourly demand gives

$$\alpha_h = \min\Big(1,\ R_h \,/\, \textstyle\sum_{j=h}^{23} \hat{y}_j^{(q)}\Big), \qquad q \in \{0.5, 0.7, 0.9\}. \tag{1}$$

- *Hindsight constant* (reference): $\alpha = \min(1, B/D)$ with the day's true demand D. It is the denominator of our primary metric. It is not an upper bound, because a time-varying rate can capture more clicks than a constant one.

Raising q in Eq. (1) inflates the planned demand and lowers α_h. This is the "hedging" knob: it trades spend for protection against early depletion. Note that the denominator is the sum of hourly quantiles, which is not the q-quantile of the remaining day's demand.

Variants used in the decomposition analyses. (i) *Scaled throttle*: the denominator of Eq. (1) is multiplied by a scale s, with the median forecast; $s < 1$ plans for less demand and spends more. We evaluate 25 values from 0.50 to 1.10, and choose s per budget level on the validation days. (ii) *Calibrated hedge*: the denominator is the median plan times a factor $f_q(h)$, the q-quantile of realised remaining-day spend divided by the median plan at origin hour h, taken over all campaigns and the earlier test days only (so it is evaluated on the last 4 test days). (iii) *Feedback*: a PID term on the gap between spend so far and a plan that spreads $\min(B, \hat{D})$ in proportion to the day-start forecast profile is added to α_h of Eq. (1), and the sum is clipped to $[0, 1]$. Its gains are chosen on the validation days with the seasonal-naive forecast, because the neural forecasts exist only for test origins. (iv) *Static plans*: the planned remaining demand at hour h is a day total times the share of the day-start forecast that lies in hours h to 23, with no re-planning; the total and the shape can come from different forecasters. (v) *Oracle*: the true remaining demand of the day, or the true day total with a forecast shape, in place of the forecast. (vi) *Dual-gradient pacer*: a reactive throttle $\alpha_h = \min(1, 1/(1+\mu_h))$ whose multiplier μ_h moves each hour by a step size times the relative gap between the last hour's spend and its target, either flat ($B/24$) or the campaign's training-day profile; the step size is chosen on the validation days. (vii) *Feedback on the hourly re-plan*: the feedback of (iii) applied to the gap between each hour's realised spend and what the previous plan expected.

4 Forecasting Models

The target is hourly campaign spend. Forecasts are issued at every hour of the test period for the next 24 hours, using only earlier hours. All models output the quantiles $\{0.1, 0.5, 0.7, 0.9\}$.

- *Seasonal naive*: empirical quantiles of the same hour on the previous seven days.
- *LightGBM*: a direct multi-horizon model with the forecast step as a feature. It is trained with the quantile objective, one model per quantile. Features are spend lags, same-hour

Table 1: Pacing outcomes on the test days. Clicks are relative to the hindsight constant-rate reference, pooled over campaign-days and replay seeds; the best non-reference value per budget level is in bold. The right block reports $\rho = 0.5$. For the neural models, values pool the training seeds. Bootstrap confidence intervals and the seed SDs for all entries are in the results file `pacing_summary.csv`.

	Clicks / reference			At $\rho = 0.5$			
Policy	ρ=0.3	ρ=0.5	ρ=0.7	Conv./ref.	BUR (%)	Early depl. (%)	PVI
Hindsight constant (ref.)	1.000	1.000	1.000	1.000	90.6	9.5	0.000
No pacing	0.985	0.979	0.988	1.018	94.4	80.1	0.000
Even pacing	0.993	0.960	0.942	0.966	90.2	6.4	0.314
PID (tuned)	1.009	0.990	0.988	1.004	92.2	31.9	0.577
Seasonal naive, q=0.5	1.007	0.989	0.983	0.988	91.3	6.4	0.083
LightGBM, q=0.5	1.019	0.997	0.992	0.998	92.4	5.0	0.094
NHITS, q=0.5	**1.028**	**1.007**	0.995	1.005	92.5	4.3	0.118
TFT, q=0.5	1.027	1.007	**0.996**	1.006	92.7	5.5	0.097
Seasonal naive, q=0.7	0.984	0.965	0.959	0.957	87.7	2.4	0.075
LightGBM, q=0.7	0.995	0.972	0.965	0.964	88.7	1.6	0.082
NHITS, q=0.7	1.011	0.985	0.971	0.976	89.1	1.1	0.101
TFT, q=0.7	1.010	0.985	0.974	0.977	89.4	1.6	0.085
Seasonal naive, q=0.9	0.930	0.910	0.904	0.897	79.6	0.6	0.070
LightGBM, q=0.9	0.914	0.892	0.887	0.879	78.2	0.5	0.071
NHITS, q=0.9	0.950	0.919	0.904	0.904	79.6	0.2	0.087
TFT, q=0.9	0.945	0.916	0.904	0.905	79.6	0.3	0.074

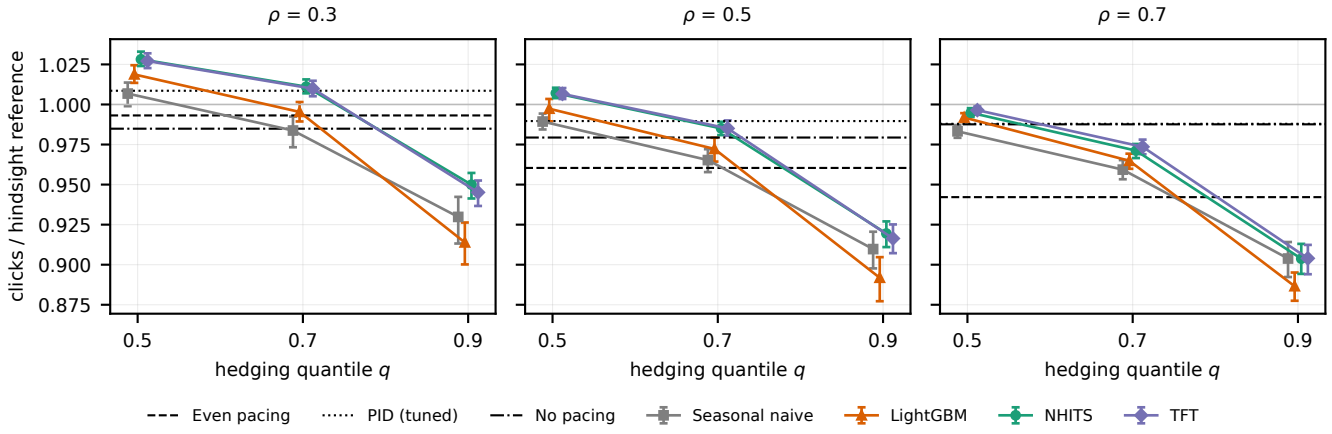


Figure 1: Clicks relative to the hindsight reference as a function of the hedging quantile, with 95% campaign-bootstrap CIs. Horizontal lines are the forecast-free policies.

seasonal lags, rolling statistics, calendar phase and static campaign statistics, with the target normalized by the campaign's training mean.

- *TFT* and *NHITS*: `neuralforecast` models with a 168-hour input window, multi-quantile loss, robust scaling, historical impressions and clicks, calendar phase as known future inputs, and static campaign statistics.

Models are trained on the training days and early-stopped on the validation days. Hyperparameters were fixed before evaluation and not tuned on test data; they are listed in the protocol (deviation log, entry on hyperparameters) and in the released code. The neural models are trained with 3 seeds each.

5 Experimental Protocol

Data. The Criteo Attribution dataset [6] contains 16,468,027 impressions from 675 campaigns over 742 hours (its documentation states 30 days), with the paid cost (transformed by Criteo), click and attributed conversion of each impression. The data are released under CC BY-NC-SA 4.0. We use the first 30 days and discard the last 22 hours. The first 21 days are for training, the next 4 for validation and the following 5 for testing. A campaign is included if it has spend in at least 90% of training hours. The inclusion rule uses training data only, but the threshold itself was chosen after a descriptive profile of the whole dataset (see Protocol below). The

Table 2: Forecast accuracy on the test origins, averaged over campaigns; ± is the standard deviation over seeds. $MASE_h$ is the MASE at forecast step h, $sPL_{0.9}$ is the 0.9-quantile pinball loss divided by the campaign's mean training hourly spend, and Cov. is the share of outcomes inside the nominal 80% interval. Time is the wall-clock time to fit and forecast (neural models on one RTX 3070 Ti, LightGBM on an Apple M4 CPU).

Forecaster	MASE	$MASE_{h=1}$	$MASE_{h=24}$	$sPL_{0.9}$	Cov. 80%	Time (s)
Seasonal naive	0.917	0.920	0.908	0.194	60.6	0
LightGBM	0.853	0.810	0.871	0.185	71.7	76
NHITS	0.794 ± 0.014	0.719	0.813	0.158	75.8	40
TFT	0.787 ± 0.004	0.729	0.814	0.161	76.9	205

rule keeps 414 campaigns, which account for 91.7% of all spend, and gives 2,070 test episodes, each replayed with 10 seeds.

Metrics. Forecast accuracy is measured with MASE (scaled by each campaign's in-sample lag-24 naive error), the scaled pinball loss sPL_q, the 80% interval coverage and the quantile-crossing rate. Pacing is measured by the ratio of clicks (and of attributed conversions) to the hindsight reference, pooled over episodes; budget utilization (BUR); the early-depletion rate (budget exhausted before hour 20); and the pacing volatility index (PVI), the RMS change of α_h between consecutive hours. For the decomposition we also report the change in total spend and in clicks per unit of spend.

Statistics. We report 95% confidence intervals (CIs) from a bootstrap over campaigns (2,000 resamples); differences between methods are paired. Pre-specified paired comparisons use Wilcoxon signed-rank tests [15] on per-campaign click ratios, with a Holm correction [7] over all 126 comparisons; these use the 409 campaigns with positive hindsight clicks (a ratio needs a positive denominator), while all other analyses use all 414. These intervals and tests treat the trained models as fixed: the 3 seeds of each neural model are pooled inside each per-campaign ratio, so training-seed variation is reported separately (Section 6.2).

Protocol. The study protocol, including a deviation log, was written before the test split was evaluated. Two things it did not anticipate are disclosed in the log: the activity threshold and other design choices were informed by a descriptive profile of the whole dataset, which includes the test hours; and before the final runs we inspected the test-origin accuracy of the seasonal-naive and LightGBM forecasts and of a short smoke-test TFT run, and ran a pacing smoke test on ten campaigns' test days. No hyperparameter was changed as a result. The log also records two widenings of the PID grid (on validation data) and intermittent GPU faults during neural training (crashed runs produced no output and were rerun). Everything in Sections 6.3 to 6.7 was added after the main results were seen: the split of gains into spend and efficiency, the scaled, calibrated, static and oracle plans, the feedback variant, and an exploratory split of campaigns by size and volatility. A model-written review of an earlier version led to the 25-scale grid, the swap and the oracle runs, and to the dual-gradient baseline, the feedback on the hourly re-plan and the iPinYou replication. The profile-target dual-gradient variant, as specified in advance, underspends, and a corrected variant was added after seeing that. Each new simulation was entered in the log, with a reading rule where a verdict was drawn, before it ran; the two analyses that only re-read saved episodes (the spend split and the group split) were logged after they ran. None changes a number in the main tables. The matched-spend comparison had no reading rule. Two metrics of the protocol are not reported here: the hourly spend deviation from the oracle profile, which is computed and stored in `pacing_summary.csv`, and the forecast metrics on the validation split. The protocol defined seasonal naive as the mean of the previous seven days, but the study ran the empirical quantiles (the median at q=0.5); the mean version is worse (Section 6.3). An independent rerun on a second machine rebuilt the data from the raw file, retuned the PID gains, regenerated the seasonal-naive and LightGBM forecasts and reproduced the pacing summary exactly; it reused the saved neural forecasts, whose checksums match, because neural training needs a GPU.

6 Results

6.1 Forecast accuracy

All four forecasters beat the in-sample seasonal-naive scale (Table 2). TFT (MASE 0.787) and NHITS (0.794) have the lowest mean MASE, followed by LightGBM (0.853) and seasonal naive (0.917). The ranking is less clear for the typical campaign: the median-campaign MASE is 0.708 for TFT, 0.723 for NHITS and 0.722 for LightGBM, and TFT has a lower MASE than LightGBM in 70.5% of campaigns, NHITS in 56.0%. We report no confidence intervals for these accuracy differences. The intervals are too narrow: the nominal 80% interval covers only 76.9% of outcomes for TFT and 60.6% for seasonal naive.

6.2 Pacing outcomes

Table 1 and Figure 1 summarize pacing. Throttling on the median forecast of a neural model is the best policy we tested at every budget level. It captures 3.4–5.4 points more clicks than even pacing and 0.7–2.0 points more than tuned PID. A seasonal-naive profile at q=0.5 already captures 1.4–4.1 points more than even pacing, and the neural models add 1.2–2.1 points on top of it. All 12 of 12 paired comparisons against even pacing and PID are significant after Holm correction (largest adjusted p = 0.00052), conditional on the trained models. The policy is also far smoother: its PVI of 0.08–0.12 compares with 0.43–0.60 for PID, and it depletes early in only 3.4–6.1% of episodes, against 12.8–31.9% for PID.

TFT and NHITS are indistinguishable. They differ by at most 0.2 points of clicks. The training-seed SD of the click ratio at q=0.5 is up to 0.45 points for NHITS and 0.18 points for TFT, so the difference is smaller than the variation between seeds of the same model. Only 1 of 3 of their paired comparisons is significant, and those tests treat the trained models as fixed. The neural models beat LightGBM by 0.3–1.0 points; the lower end is below the NHITS seed SD, and LightGBM and seasonal naive were run once, so their seed variation is unknown. In an exploratory split of the 414 campaigns into terciles of size and of volatility, both measured on training hours, the gain of TFT over the seasonal-naive profile was positive in all 18 group-and-budget cells (1.0 to 3.1 points) and larger in the most volatile third at the two tighter budgets; the intervals overlap and are not corrected for the number of cells.

Table 3: Split of the click gain at the median plan (points of the hindsight reference, with 95% campaign-bootstrap intervals; spend and clicks per unit of spend as relative changes). "Scaled naive" is the seasonal-naive forecast total times the scale chosen on the validation days (25-value grid). Intervals are shown for the click column only.

Comparison	ρ	Clicks (points)	Spend (%)	Clicks per spend (%)
TFT − seasonal naive	0.3	2.03 [1.57, 2.62]	1.9	0.15
	0.5	1.73 [1.36, 2.11]	1.8	−0.03
	0.7	1.32 [1.06, 1.63]	1.4	−0.03
Scaled naive − seasonal naive	0.3	0.73 [0.55, 0.93]	1.4	−0.69
	0.5	1.03 [0.82, 1.25]	1.9	−0.83
	0.7	1.24 [0.96, 1.53]	2.4	−1.12
TFT − scaled naive	0.3	1.30 [0.93, 1.77]	0.4	0.84
	0.5	0.69 [0.39, 1.03]	−0.1	0.81
	0.7	0.08 [−0.19, 0.37]	−1.0	1.11

6.3 Where the gain comes from

Delivery, not efficiency. Table 3 splits the click gain into budget spent and clicks per unit of spend. TFT's gain over the seasonal-naive profile comes with 1.4–1.9% more spend and no loss of clicks per unit of spend (−0.03 to 0.15%, with an interval that includes zero at 3 of 3 budget levels). NHITS behaves the same way (spend 1.2–1.9%). A plain rescaling of the naive plan buys its clicks differently: it spends 1.4–2.4% more and changes clicks per unit of spend by −0.69 to −1.12%. Against tuned PID the picture differs: TFT spends about the same (−0.3 to −0.1%) and buys 1.1–2.1% more clicks per unit of spend, with the interval above zero at 3 of 3 levels. A forecast therefore helps against reactive control through efficiency, and against a seasonal profile through delivery: the seasonal-naive plan under-spends, and the learned forecast corrects that without paying for it in efficiency.

A tuned dual-gradient pacer does not close the gap. Tuned PID may not be the strongest reactive reference, so we also tuned a dual-gradient throttle on the validation days. With a flat hourly target it differs from tuned PID by −4.8 to −3.0 points; TFT is ahead of it by 3.9–6.7 points and the scaled naive plan by 3.81–5.39, with every interval above zero, so by the pre-registered rule the forecast advantage survives. The profile-target variant, as specified, under-spends (−11.6 to −9.7 points against PID). A corrected variant that divides the gap by $B/24$, added after seeing this, is still −2.6 to −1.1 points from tuned PID, and TFT is ahead of it by 2.9–3.6 points. Tuned PID remains the strongest reactive pacer we tried.

One scale recovers much of it. If the gain over the profile is mostly delivery, one scale on the naive total should capture much of it. The scales chosen on validation days are 0.85, 0.825 and 0.775 for the tight, medium and loose budgets, none at the edge of the grid. The scaled plan gains 0.73 ([0.55, 0.93]), 1.03 ([0.82, 1.25]) and 1.24 ([0.96, 1.53]) points over the plain naive plan: 36%, 60% and 94% of the TFT gain. TFT stays ahead of the scaled plan at the tight (1.30, [0.93, 1.77]) and medium (0.69, [0.39, 1.03]) budgets and ties it at the loose one (0.08, [−0.19, 0.37]). By the pre-registered rule the total does not explain almost all of the gain (the scaled plan reaches 80% of the TFT gain only at the loose budget), and an edge beyond one scale remains. The scaled plan beats tuned PID at the medium budget (1.00, [0.32, 1.79]). Using the mean of the previous seven days instead of the median, as the protocol first specified, changes clicks by −0.73 to −0.95 points relative to the median plan.

Total or shape? The throttle uses only the sum of the remaining forecast, so we separate the two on the plans of the forecast issued at hour 0 (static plans, no re-planning). With the plan (naive total, naive shape) as the base, the plan (TFT total, TFT shape) gains 1.52, 1.35 and 1.07 points. Keeping the TFT total and the naive shape gains 1.25 ([0.82, 1.83]), 1.02 ([0.71, 1.38]) and 0.72 ([0.50, 0.95]); keeping the naive total and the TFT shape gains only 0.16 ([−0.02, 0.31]), 0.17 ([0.05, 0.29]) and 0.25 ([0.14, 0.37]). By the pre-registered rule the total is the source at the tight budget; at the medium and loose budgets the result is partial: the total supplies 76% and 68% of the gain and the shape adds a small positive amount. Re-planning every hour adds 0.25–0.51 points to TFT and nothing to seasonal naive, whose forecast does not depend on the origin hour.

Headroom. With the true remaining demand in place of the forecast, the same throttle gains 1.03 ([0.73, 1.35]), 1.50 ([1.23, 1.78]) and 1.54 ([1.27, 1.81]) points over TFT, and 3.06, 3.22 and 2.86 over seasonal naive. This is moderate headroom by the pre-registered thresholds (below 1 point would be little, 2 points or more a lot). A true day total with the naive shape gains 1.44–1.84 points over the static naive plan, and with the TFT shape 0.63–0.86 over the static TFT plan. True remaining demand then adds 0.91, 1.00 and 1.04 points by also knowing the hourly path.

An edge at matched spend (exploratory). Scaling changes how much is spent, so we also compare each method with the curve that the 25 scales trace in (spend, clicks) space, at the method's own spend. TFT lies 1.20 ([0.91, 1.56]), 0.74 ([0.51, 1.01]) and 0.47 ([0.30, 0.66]) points above the curve at the tight, medium and loose budgets, and NHITS 1.30 ([0.99, 1.75]), 0.83 ([0.58, 1.12]) and 0.44 ([0.24, 0.64]). LightGBM lies on it at the medium and loose budgets (0.01, [−0.60, 0.48]; 0.05, [−0.14, 0.25]) and above it at the tight one (0.45, [0.13, 0.77]); PID lies below it. This comparison had no reading rule and is descriptive; its intervals, like all others here, ignore training-seed variation.

6.4 Hedging

Hedging costs a lot. Clicks fall monotonically as q rises from 0.5 to 0.9 in 12 of 12 forecaster and budget combinations, and moving to q=0.9 costs 7.7–10.6 points. All 24 of 24 comparisons of q=0.9 policies against even pacing and PID favor the baselines (24 are significant). This environment barely penalizes early depletion: no pacing (click ratio 0.988) ties tuned PID (0.988) at the loose budget (Table 1), so part of the hedging cost is a property of the simulator. Figure 2 shows what it buys: early depletion almost disappears (0.1–0.7% of episodes), but budget utilization falls to 75.2–82.8%. Relative to even pacing, a TFT plan at q=0.9 changes spend by −7.4 to −5.7% and clicks per unit of spend by 1.8 to 2.8%, so the cost of hedging is under-delivery.

Summing hourly quantiles over-covers. Eq. (1) divides by the sum of hourly quantiles. That a sum of quantiles is not the quantile of a sum is textbook; what we report is how large the gap is here. For single hours the 90th-percentile forecast is well calibrated (TFT: 90.8% of hours at or below it), but the realised remaining-day spend was at or below the summed plan in 94.0–97.4% of cases (95.9–97.4% for the three learned forecasters), and at the 70% level in 76.4–80.1%.

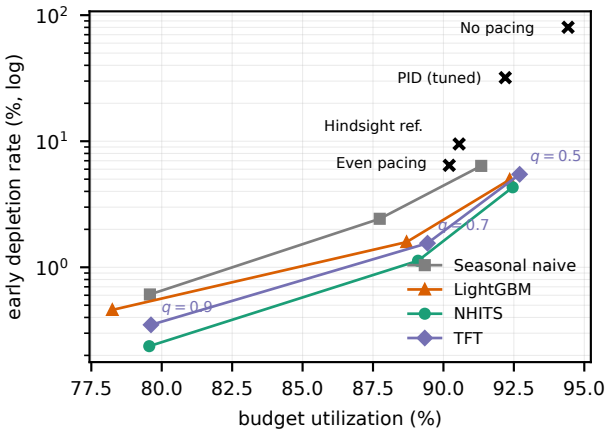


Figure 2: Delivery trade-off at $\rho = 0.5$: budget utilization against the early-depletion rate (log scale). Lines connect $q = 0.5, 0.7, 0.9$ of each forecaster.

The 90% plan is 1.80–2.06 times the median plan, where 1.62–1.70 would give 90%. The spread of the daily total is almost the same for all four forecasters.

A calibrated daily-total hedge recovers only part of the loss. Replacing the sum by the median plan times a factor learned on earlier test days improves TFT at q=0.9 by 1.54 ([0.81, 2.46]), 1.21 ([0.42, 2.15]) and 0.95 ([0.10, 2.00]) points, but this recovers only 20%, 14% and 11% of the loss that the summed plan suffers against the median plan (days 26 to 29 only). Most of the cost of hedging at 0.9 therefore remains after calibration. At q=0.7 the calibrated plan recovers 85%, 55% and 37% of the loss, and at the tight budget costs only −0.23 points ([−0.41, −0.05]) against the median plan. For seasonal naive the calibrated 90% plan is worse than the summed one (−3.6 to −1.9 points). A cross-fitted variant that borrows the factor from a random half of the other campaigns did no better for TFT at 0.9 (−0.28 to 0.04 points); in this simulator any factor above 1 only lowers clicks, so better coverage does not mean more clicks. No pacing sits at the opposite corner of Figure 2: it depletes early in 59.1–92.1% of episodes, and it captures 1.4–3.2 points more attributed conversions than the reference while capturing fewer clicks, so an operator who optimizes for conversions should not assume that click-optimal pacing is also conversion-optimal.

6.5 Adding feedback

Adding the PID correction of Section 3 to the forecast throttle helps every forecast at the tight budget: seasonal naive by 2.22 ([1.65, 2.85]) points, scaled naive by 1.54 ([0.98, 2.13]), TFT by 0.50 ([0.23, 0.79]) and NHITS by 0.49 ([0.14, 0.83]). By the pre-registered rule, feedback does not help TFT: it helps at the tight budget and hurts at the medium and loose ones (−0.97, [−1.68, −0.41]; −2.01, [−2.79, −1.43]), and the neural edge does not survive feedback. TFT with feedback minus scaled naive with feedback is 0.26 ([−0.02, 0.59]), 0.38 ([−0.31, 1.42]) and −1.92 ([−2.75, −1.26]) points at the tight, medium and loose budgets: the intervals include zero at the first two and lie below it at the third. A variant that tracks the previous hour's plan instead of the plan of hour 0, since the throttle re-plans every hour, gives the same verdicts: feedback minus no feedback for TFT is 0.14 ([−0.02, 0.29]), −0.12 ([−0.34, 0.10]) and −1.64 ([−2.04, −1.29]) points, and TFT with that feedback minus scaled naive with it is −0.19, 0.27 and −1.39; for seasonal naive at the tight budget it adds 1.78 ([1.30, 2.37]) points. Two cautions apply. The neural models use the gains tuned for seasonal naive (proportional gain 8, 32 and 8; the grid edge was reached at $\rho = 0.5$), so part of the loss may be a mismatch. And gains tuned on four validation days are unstable: for seasonal naive at the loose budget feedback added 1.0 points on the validation days and changed the test result by −0.82 ([−2.50, 0.17]).

6.6 A second environment: a synthetic audience world

To see whether the pattern depends on the logging policy and the transformed costs of one dataset, we asked the same question in a world builder developed for this study. It generates a population of 50,000 users with latent interests and churn (mean lifetime 14 days), 40 campaigns and 28 days with daily and weekly rhythms, several cost models, and replays the opportunity stream against a Lagrangian hindsight upper bound (UB). The pacers use the true click probabilities, so the world isolates pacing from click prediction. At the realistic setting, over 6 worlds and 4 cost models, a reactive controller captures 65.8% of UB for flat prices. A plain replay of the previous seven days' opportunities gains 1.5–2.7 points over it, and rescaling that replay to the true expected traffic adds only 0.1–0.3 points more: the best forecast-driven pacer is 1.6–3.0 points ahead of the reactive one, and only with time-of-day prices is that interval clear of zero (1 of 4 cost models). The ordering matches the Criteo result, in which looking ahead beats reacting and the total carries most of the gain. The world is a controlled check, not evidence about real traffic: its rhythms are planted, and its daily totals vary far less than those of real campaigns.

6.7 A second real dataset: iPinYou

We repeated the comparison on the iPinYou season-2 training logs: 12.2 million won impressions of 5 advertisers over 7 days, with paying prices and clicks joined from the click logs. A campaign is an advertiser, days 1 to 4 are training, day 5 validation and days 6 and 7 test, so there are 10 test episodes; the forecast is seasonal naive over the previous 3 days and no neural model is trained. The result is descriptive, and by the rule fixed beforehand it is inconsistent with Criteo: the ordering (scaled naive at least the naive throttle, and the naive throttle at least even pacing and PID) holds at 0 of 3 budget levels. The 3-day naive throttle is no better than even pacing (−1.1, −1.1 and −0.8 points; every interval includes zero), and no pacing is the best realistic policy at the tight budget (click ratio 0.973 against 0.951 for the scaled plan). A scale on the naive total gains at least as much as on Criteo (1.3, 2.0 and 1.3 points), but the intervals include zero. The only clear difference is the oracle, which is 6.4, 5.5 and 4.9 points ahead of the naive throttle ([1.6, 15.1]; [1.8, 10.8]; [1.3, 10.5]). So on this dataset a replay of the previous days is a poor forecast and a good forecast would be worth a lot, but we cannot say which forecaster would deliver it: with 5 advertisers, 10 episodes, one validation day for tuning and a scale at the grid

edge at the loose budget, the replication supports neither the Criteo ordering nor a contradiction of it by more than a point estimate.

7 Discussion

What neural predictors add. Forecast-driven throttling beat even pacing and a tuned PID here. Relative to a seasonal profile, the learned forecaster's contribution is mostly that it delivers more of the budget without losing efficiency: one scale on the naive total recovers 36% to 94% of the gain, more as the budget loosens, and in a total/shape swap the forecast total supplies 68–82% of it. A smaller edge remains beyond one scale, and an oracle forecast would add 1.03–1.54 points over TFT, so better forecasts still have room to help in this simulator. A cheaper model such as NHITS (40 s versus 205 s for TFT) was indistinguishable from TFT.

Practical guidance in this setting. (i) If a deep model is not available, scale the seasonal-naive total with a factor chosen on validation days; it recovers much of the neural gain. (ii) Plan against the median rather than the summed upper quantile; at 0.9 the loss is mostly under-delivery that calibration does not remove, and the calibrated 70% plan also costs clicks against the median (−0.23 to −1.37 points). (iii) Add feedback only at tight budgets, and do not tune its gains on a few days. (iv) Forecast accuracy gains shrink when they reach pacing, plausibly because the throttle depends only on the sum of the remaining forecasts, where hourly errors partly cancel; we did not test this.

Limitations. The log contains only impressions that the logging bidder won under its own bidding and pacing, so the replayed demand is conditional on that policy. The costs are transformed by Criteo. The throttling model ignores competitive responses and price effects, and, as the comparison of no pacing with tuned PID shows, barely penalizes early depletion. The test period covers 5 days of one real dataset, and the causal calibration is evaluated on the last 4 of them. The day boundary is relative to the dataset start, not local midnight. The tests condition on the trained models, and LightGBM and seasonal naive were run once. The scale, the feedback gains and the PID gains were chosen on four validation days; the PID gains lie on the edge of the search grid (for example K_p=0.03125, K_i=1, K_d=32 at ρ=0.5) and its high PVI suggests an under-damped controller, so a better-designed feedback controller, such as a dual-gradient pacer, may narrow the gap. A variant that tracks the hourly re-plan gives the same feedback verdicts. The decomposition, hedging, feedback, oracle and group analyses are post hoc, the matched-spend comparison had no reading rule, and the efficiency change between TFT and the scaled plan has no interval. The hindsight reference is not a ceiling, and the oracle uses logged demand. The second real dataset (iPinYou) is small (5 advertisers, 10 test episodes, a 3-day forecast, one validation day for tuning) and does not reproduce the Criteo ordering; the synthetic world supports the ordering, not the sizes. With hundreds of campaigns, small differences become statistically significant, so we emphasize effect sizes.

8 Conclusion

On real campaign logs and within the throttling model studied, forecast-driven throttling improved daily budget pacing over even pacing and tuned PID. Over a seasonal profile, what a learned forecaster adds is mostly a better daily total, delivered as extra spend without losing efficiency: a single validated scale on the naive total recovers 36% to 94% of the gain, the forecast total supplies 68–82% of it in a total/shape swap, and a smaller edge (0.47–1.20 points for TFT at matched spend, exploratory) remains for TFT and NHITS. An oracle forecast would add 1.03–1.54 points. Hedging at the 90th percentile costs the most; summing hourly quantiles over-covers the day, but a calibrated daily-total hedge recovers only part of the loss, and part of the cost is a property of a simulator that barely penalizes early depletion. Feedback on top of the forecast helps at the tight budget only. A synthetic world gives the same ordering. TFT and NHITS were indistinguishable. A tuned dual-gradient pacer trails tuned PID and forecast throttling stays ahead of it, but a small replication on iPinYou does not reproduce the ordering. Future work should add seed-aware inference, a second real dataset long enough to train deep forecasters, and bid-landscape-aware simulators.

References

[1] Deepak Agarwal, Souvik Ghosh, Kai Wei, and Siyu You. 2014. Budget Pacing for Targeted Online Advertisements at LinkedIn. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. doi:10.1145/2623330.2623366

[2] Santiago Balseiro, Anthony Kim, Mohammad Mahdian, and Vahab Mirrokni. 2017. Budget Management Strategies in Repeated Auctions. In *Proceedings of the 26th International Conference on World Wide Web (WWW)*. 15–23.

[3] Cristian Challu, Kin G. Olivares, Boris N. Oreshkin, Federico Garza Ramirez, Max Mergenthaler-Canseco, and Artur Dubrawski. 2023. NHITS: Neural Hierarchical Interpolation for Time Series Forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. doi:10.1609/aaai.v37i6.25854

[4] Yuanlong Chen. 2025. A Practical Guide to Budget Pacing Algorithms in Digital Advertising. arXiv:2503.06942.

[5] Liang Dai, Kejie Lyu, Chengcheng Zhang, Guangming Zhao, Zhonglin Zu, Liang Wang, and Bo Zheng. 2023. Percentile Risk-Constrained Budget Pacing for Guaranteed Display Advertising in Online Optimization. arXiv:2312.06174.

[6] Eustache Diemert, Julien Meynet, Pierre Galland, and Damien Lefortier. 2017. Attribution Modeling Increases Efficiency of Bidding in Display Advertising. In *Proceedings of the AdKDD and TargetAd Workshop, KDD.* arXiv:1707.06409.

[7] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70.

[8] Rob J. Hyndman and Anne B. Koehler. 2006. Another Look at Measures of Forecast Accuracy. *International Journal of Forecasting* 22, 4 (2006), 679–688.

[9] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems 30 (NIPS).*

[10] Roger Koenker and Gilbert Bassett. 1978. Regression Quantiles. *Econometrica* 46, 1 (1978), 33–50.

[11] Kuang-Chih Lee, Ali Jalali, and Ali Dasdan. 2013. Real Time Bid Optimization with Smooth Budget Delivery in Online Advertising. In *Proceedings of the Seventh International Workshop on Data Mining for Online Advertising (ADKDD).*

[12] Bryan Lim, Sercan Ö. Arık, Nicolas Loeff, and Tomas Pfister. 2021. Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting. *International Journal of Forecasting* 37, 4 (2021), 1748–1764.

[13] Kin G. Olivares, Cristian Challú, Federico Garza, Max Mergenthaler Canseco, and Artur Dubrawski. 2022. NeuralForecast: User Friendly State-of-the-Art Neural Forecasting Models. PyCon, Salt Lake City, Utah, US. https://github.com/Nixtla/neuralforecast.

[14] Prashant Shekhar and Caroline Howard. 2026. Decision-Calibrated Conformal Uncertainty for Pacing Decisions in Streaming Advertising. arXiv:2606.10187.

[15] Frank Wilcoxon. 1945. Individual Comparisons by Ranking Methods. *Biometrics Bulletin* 1, 6 (1945), 80–83.

[16] Jian Xu, Kuang-Chih Lee, Wentong Li, Hang Qi, and Quan Lu. 2015. Smart Pacing for Effective Online Ad Campaign Optimization. In *Proceedings of the 21st ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD).*

[17] Weinan Zhang, Yifei Rong, Jun Wang, Tianchi Zhu, and Xiaofan Wang. 2016. Feedback Control of Real-Time Display Advertising. In *Proceedings of the ACM International Conference on Web Search and Data Mining (WSDM).* arXiv:1603.01055.

[18] Weinan Zhang, Shuai Yuan, and Jun Wang. 2014. Optimal Real-Time Bidding for Display Advertising. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD).*